

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_58a19e3e-3ce\agent-  
a5035de6a761c0f9d.jsonl`  
- SHA-256: `41e45f9721ea043b331597ba9fcfe4ce5435ca3c74e2e3992eafa1020f8d4a57`  
- Source modified: `2026-07-06T23:21:10+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_58a19e3e-3ce`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T23:16:54.905Z)

```
Reviewing GitHub PR #54 "feat(policy): add F5 deep inspection budget boundary" (branch  
feat/f5-deep-inspection-budget -> main). ADR 0012 F5.  
+364/-4, 3 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr54 (read there or 'git -C  
C:/Users/avidu/Projects/Telegram show origin/feat/f5-deep-inspection-budget:<path>').  
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr54/src  
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.
```

WHAT F5 DOES (policy.py) ? BOUNDARY ONLY, no downloader/ffprobe/hash/scan (explicitly deferred):

```
- PolicyBudget(frozen dataclass): max_bytes_per_item, timeout_s, max_concurrent, + private  
_semaphore built in __post_init__ (via object.__setattr__)  
  when max_concurrent>0. async inspect(fetcher, content_item): 'async with self._semaphore:  
return await asyncio.wait_for(fetcher.inspect(item,self), timeout=timeout_s)'.  
- ContentFetcher(Protocol): async inspect(content_item, budget)->StageResult.  
PolicyContext.fetcher/budget now typed (were object|None in F1).  
- load_deep_inspection_enabled(policy): deep_inspect bool OR {enabled: bool}; invalid ->  
ValueError -> stage fail-OPEN (pass, invalid_deep_inspection_config).  
- DeepInspectionStage (Tier 4, LAST after topic/url/caption): opt-out if not enabled -> pass.  
_deep_inspection_budget_result checks (in order, all  
  BEFORE calling fetcher): budget present, budget valid (max_bytes/timeout/max_concurrent all  
>0), content_item.size_bytes present, size<=max_bytes ->  
  returns a FAIL StageResult on any violation. Then if fetcher missing -> fail. Else  
budget.inspect(...) under semaphore+wait_for; TimeoutError -> fail 'budget exceeded'.  
- Live wiring (_evaluate_policy_stages) creates PolicyContext WITHOUT fetcher/budget (both  
None), so deep_inspect enabled -> stage fails 'budget missing'  
  -> video DROPPED (fail-closed). This is intentional/dormant (no /set deep_inspect command  
exists).
```

VERIFIED by main reviewer: gate green (543 passed @ 94.57%); budget checks run BEFORE the
fetcher; deep_inspect enabled-without-budget drops the item
(fail-closed, safe direction); default (deep_inspect unset) unaffected. Comprehensive tests
(missing/invalid budget, missing size, size-exceeded, timeout,
concurrency max_active==1, missing fetcher, valid delegate).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No
praise. Concrete failure scenario, ranked.
Severity: high (budget/timeout/concurrency BYPASS ? heavy work runs unbounded, or a real bug),
medium (real latent/design), low (note/hygiene/intentional-fail-closed).

DIMENSION: config parsing, fail posture, wiring, and cross-phase dependency.

Investigate:

```
- load_deep_inspection_enabled: deep_inspect bool | {enabled: bool}; other shapes -> ValueError  
-> fail-OPEN (pass). But an ENABLED-but-unbudgeted item ->  
  fail-CLOSED (drop). Is the open/closed split coherent (parse-error=ignore vs asked-but-cant-
```

run=refuse)? Any input that is 'enabled' yet slips through as pass?
 e.g. deep_inspect={'enabled': 'true'} (string) -> raises -> fail-open (so a stringly-typed enable silently does NOT enable). Confirm and assess.
 - OPERATIONAL footgun (confirmed): enabling deep_inspect today drops ALL matched videos (no fetcher/budget wired). It is dormant + fail-closed-safe, but a
 future /set deep_inspect command MUST NOT ship before the fetcher/budget wiring. Is this documented anywhere (comment/ADR)? Severity as a note.
 - Stage order topic->url->caption->deep-inspection via evaluate_pipeline; deep-inspection LAST (most expensive) - correct short-circuit? default (unset) is a no-op pass?
 - Does this PR read/depend on content_item.size_bytes (populated today?) ? size_bytes IS set on real videos (metadata), so the size budget works on live data. Confirm.
 - Any interaction with F2/F3/F4 stages sharing the same config object? deep_inspect is a top-level key (not under security_checks) - confirm no collision.
 Report findings.

2. assistant (2026-07-06T23:16:57.406Z)

I'll review PR #54. Let me start by reading the changed files.

3. user (2026-07-06T23:16:59.674Z)

```
49f15a8 feat(policy): add deep inspection budget boundary
a7cbbda feat(policy): add URL reputation stage (#53)
5cd65d1 feat(policy): add caption safety stage (#52)
6d3d7a6 feat(policy): add F2 topic preferences (#51)
fb2d2ea feat(policy): add F1 pipeline primitives (#50)
---FILES---
src/tg_video_filter/policy.py      | 144 ++++++
src/tg_video_filter/telethon_client.py | 8 +-
tests/test_policy.py              | 216 ++++++
3 files changed, 364 insertions(+), 4 deletions(-)
```

4. user (2026-07-06T23:16:59.869Z)

src\tg_video_filter\policy.py

5. assistant (2026-07-06T23:17:01.731Z)

Let me read the full policy.py diff and the key source file.

6. user (2026-07-06T23:17:04.279Z)

```
diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
index be60459..879b28d 100644
--- a/src/tg_video_filter/policy.py
+++ b/src/tg_video_filter/policy.py
@@ -7,6 +7,7 @@ the first failure. Later Track F phases add concrete stages.

from __future__ import annotations

+import asyncio
+import json
+import logging
+import re
@@ -39,6 +40,43 @@ _URL_RE = re.compile(
    _TRAILING_URL_PUNCTUATION = ".,!?;:"

+@dataclass(frozen=True)
+class PolicyBudget:
+    """Bounded execution budget for opt-in deep inspection."""
+    +
+    + max_bytes_per_item: int
+    + timeout_s: float
```

```

+     max_concurrent: int
+     _semaphore: asyncio.Semaphore | None = field(
+         default=None,
+         init=False,
+         repr=False,
+         compare=False,
+     )
+
+     def __post_init__(self) -> None:
+         if self.max_concurrent > 0:
+             object.__setattr__(self, "_semaphore", asyncio.Semaphore(self.max_concurrent))
+
+     async def inspect(self, fetcher: ContentFetcher, content_item: ContentItem) -> StageResult:
+         """Run ``fetcher`` under the timeout and concurrency budget."""
+         if self._semaphore is None:
+             raise ValueError("max_concurrent must be positive")
+         async with self._semaphore:
+             return await asyncio.wait_for(
+                 fetcher.inspect(content_item, self),
+                 timeout=self.timeout_s,
+             )
+
+
+class ContentFetcher(Protocol):
+    """Fetcher used only by opt-in Tier 4 policy stages."""
+
+    async def inspect(self, content_item: ContentItem, budget: PolicyBudget) -> StageResult:
+        """Inspect content under ``budget`` and return a policy result."""
+        ...
+
+
+@dataclass(frozen=True)
+class PolicyContext:
+    """Immutable snapshot available to a policy stage."""
+
+@@ -46,8 +84,8 @@ class PolicyContext:
+     content_item: ContentItem
+     source: Source
+     policy: Policy
+     - fetcher: object | None = None
+     - budget: object | None = None
+     + fetcher: ContentFetcher | None = None
+     + budget: PolicyBudget | None = None
+
+
+@dataclass(frozen=True)
+@@ -253,6 +291,21 @@ def load_domain_reputation_config(policy: Policy) ->
DomainReputationConfig:
+     return parse_domain_reputation_config(decoded)
+
+
+def load_deep_inspection_enabled(policy: Policy) -> bool:
+    """Return whether Tier 4 deep inspection is explicitly enabled."""
+    decoded = json.loads(policy.config_json)
+    if not isinstance(decoded, dict):
+        raise ValueError("policy config_json must decode to a JSON object")
+    raw_deep_inspect = decoded.get("deep_inspect", False)
+    if isinstance(raw_deep_inspect, bool):
+        return raw_deep_inspect
+    if isinstance(raw_deep_inspect, dict):
+        enabled = raw_deep_inspect.get("enabled", False)
+        if isinstance(enabled, bool):
+            return enabled
+    raise ValueError("deep_inspect must be a boolean or an object with boolean enabled")
+
+

```

```

def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
    if value is None:
        return ()
@@ -618,6 +671,89 @@ def _caption_safety_metadata(
    return metadata

+class DeepInspectionStage:
+    """ADR 0012 Tier 4 opt-in boundary for budgeted heavy inspection."""
+
+    stage_name = "deep-inspection"
+
+    async def evaluate(self, ctx: PolicyContext) -> StageResult:
+        try:
+            enabled = load_deep_inspection_enabled(ctx.policy)
+        except (json.JSONDecodeError, ValueError) as exc:
+            log.warning(
+                "deep inspection disabled due to invalid config policy_id=%s",
+                ctx.policy.id,
+                exc_info=True,
+            )
+            return StageResult.pass_(
+                self.stage_name,
+                f"invalid deep inspection config ignored: {exc}",
+                invalid_deep_inspection_config=True,
+            )
+        if not enabled:
+            return StageResult.pass_(self.stage_name, "deep inspection disabled")
+
+        budget_result = _deep_inspection_budget_result(ctx)
+        if budget_result is not None:
+            return budget_result
+
+        if ctx.fetcher is None:
+            return StageResult.fail(
+                self.stage_name,
+                "deep inspection fetcher missing",
+                missing_fetcher=True,
+            )
+
+        assert ctx.budget is not None
+        try:
+            return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
+        except TimeoutError:
+            return StageResult.fail(
+                self.stage_name,
+                "deep inspection budget exceeded",
+                budget_exceeded=True,
+                exceeded_budget="timeout_s",
+                timeout_s=ctx.budget.timeout_s,
+            )
+
+def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
+    budget = ctx.budget
+    if budget is None:
+        return StageResult.fail(
+            DeepInspectionStage.stage_name,
+            "deep inspection budget missing",
+            missing_budget=True,
+        )
+    if budget.max_bytes_per_item <= 0 or budget.timeout_s <= 0 or budget.max_concurrent <= 0:
+        return StageResult.fail(
+            DeepInspectionStage.stage_name,
+            "deep inspection budget invalid",

```

```

+         invalid_budget=True,
+         max_bytes_per_item=budget.max_bytes_per_item,
+         timeout_s=budget.timeout_s,
+         max_concurrent=budget.max_concurrent,
+     )
+     size_bytes = ctx.content_item.size_bytes
+     if size_bytes is None:
+         return StageResult.fail(
+             DeepInspectionStage.stage_name,
+             "deep inspection budget cannot be enforced without content size",
+             missing_size_bytes=True,
+             max_bytes_per_item=budget.max_bytes_per_item,
+         )
+     if size_bytes > budget.max_bytes_per_item:
+         return StageResult.fail(
+             DeepInspectionStage.stage_name,
+             "deep inspection budget exceeded",
+             budget_exceeded=True,
+             exceeded_budget="max_bytes_per_item",
+             size_bytes=size_bytes,
+             max_bytes_per_item=budget.max_bytes_per_item,
+         )
+     return None
+
+
+ async def evaluate_pipeline(
+     ctx: PolicyContext,
+     stages: list[PolicyStage],
+
@@ -635,9 +771,12 @@ async def evaluate_pipeline(
__all__ = [
+     "CaptionSafetyConfig",
+     "CaptionSafetyStage",
+     "ContentFetcher",
+     "DeepInspectionStage",
+     "DomainReputationConfig",
+     "ExtractedUrl",
+     "PolicyContext",
+     "PolicyBudget",
+     "PolicyStage",
+     "StageResult",
+     "TopicPreferenceStage",
@@ -646,6 +785,7 @@ __all__ = [
+     "evaluate_pipeline",
+     "extract_caption_urls",
+     "load_caption_safety_config",
+     "load_deep_inspection_enabled",
+     "load_domain_reputation_config",
+     "load_topic_preferences",
+     "normalize_domain",

```

7. assistant (2026-07-06T23:17:06.357Z)

Now the `telethon_client.py` diff and the test diff to understand wiring.

8. user (2026-07-06T23:17:08.732Z)

```
diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index 3c7d534..2d5cbb1 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -46,6 +46,7 @@ from tg_video_filter.models import (
 )
 from tg_video_filter.policy import (
     CaptionSafetyStage,
+    DeepInspectionStage,
```

```

    PolicyContext,
    StageResult,
    TopicPreferenceStage,
@@ -439,7 +440,12 @@ async def _evaluate_policy_stages(
    return True, []
    return await evaluate_pipeline(
        PolicyContext(content_item=content_item, source=source, policy=policy),
-        [TopicPreferenceStage(), UrlReputationStage(), CaptionSafetyStage()],
+        [
+            TopicPreferenceStage(),
+            UrlReputationStage(),
+            CaptionSafetyStage(),
+            DeepInspectionStage(),
+        ],
    )

```

9. assistant (2026-07-06T23:17:12.006Z)

Now let me look at the surrounding context - the `asyncio.wait\_for` timeout exception handling, and how `size\_bytes` is populated. Let me check the `wait\_for`/`TimeoutError` relationship and `ContentItem`.

10. user (2026-07-06T23:17:14.355Z)

```

Python 3.13.13
src/tg_video_filter/backfill.py:194: meta.size_bytes is not None
src/tg_video_filter/backfill.py:202: meta.size_bytes,
src/tg_video_filter/backfill.py:212: meta.size_bytes,
src/tg_video_filter/backfill.py:274: size_bytes=meta.size_bytes,
src/tg_video_filter/backfill.py:295: size_bytes=meta.size_bytes,
src/tg_video_filter/db.py:77: size_bytes INTEGER,
src/tg_video_filter/db.py:123: size_bytes INTEGER,
src/tg_video_filter/db.py:272: size_bytes: int | None = None,
src/tg_video_filter/db.py:302: if None not in (size_bytes, duration_s, height, width):
src/tg_video_filter/db.py:303: return
f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
src/tg_video_filter/db.py:464: size_bytes INTEGER,
src/tg_video_filter/db.py:473: "height, size_bytes, mime_type, matched, forwarded,
seen_at",
src/tg_video_filter/db.py:565: document_id, duration_s, width, height, size_bytes,
mime_type,
src/tg_video_filter/db.py:574: v.duration_s, v.width, v.height, v.size_bytes,
v.mime_type,
src/tg_video_filter/db.py:604: 'tg:fp:' || size_bytes || ':' || duration_s || ':'
src/tg_video_filter/db.py:607: AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
src/tg_video_filter/db.py:613: AND size_bytes IS NOT NULL AND duration_s IS NOT
NULL
src/tg_video_filter/db.py:616: GROUP BY account_id, size_bytes, duration_s, height,
width
src/tg_video_filter/db.py:876: size_bytes=row["size_bytes"],
src/tg_video_filter/db.py:1184: size_bytes, mime_type, dedup_key, matched, seen_at)
src/tg_video_filter/db.py:1199: item.size_bytes,
src/tg_video_filter/db.py:1536: size_bytes: int,
src/tg_video_filter/db.py:1561: AND ci.size_bytes = ?
src/tg_video_filter/db.py:1566: (user_id, size_bytes, duration_s, height, width),
src/tg_video_filter/db.py:1585: size_bytes=row["size_bytes"],
src/tg_video_filter/db.py:1686: size_bytes=video.size_bytes,
src/tg_video_filter/db.py:1693: size_bytes=video.size_bytes,
src/tg_video_filter/db.py:1744: size_bytes=item.size_bytes,
src/tg_video_filter/db.py:1828: size_bytes=r["size_bytes"],
src/tg_video_filter/delivery.py:51: def _format_size(size_bytes: int | None) -> str:
src/tg_video_filter/delivery.py:53: if size_bytes is None:
src/tg_video_filter/delivery.py:55: mib = size_bytes / (1024 * 1024)
src/tg_video_filter/delivery.py:73: size_bytes: int | None,
src/tg_video_filter/delivery.py:88: f"? {_format_height(height)} .

```

```
{_format_duration(duration_s)} · {_format_size(size_bytes)}"
src/tg_video_filter/filters/base.py:26:     size_bytes: int | None = None
src/tg_video_filter/filters/size.py:14:     - If ``VideoMeta.size_bytes`` is ``None`` (unknown),
the rule fails.
src/tg_video_filter/filters/size.py:23:         if meta.size_bytes is None:
src/tg_video_filter/filters/size.py:26:         if meta.size_bytes > max_bytes:
src/tg_video_filter/filters/size.py:27:             mib = meta.size_bytes / _MIB
src/tg_video_filter/filters/size.py:29:         return FilterResult.pass_(f"size
{meta.size_bytes}B <= max {self.max_size_mb}MiB")
```

11. user (2026-07-06T23:17:14.853Z)

```
149:class ContentItem(BaseModel):
179:     size_bytes: int | None = None
407:     size_bytes: int | None = None
421:     """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height, width).
425:     if None in (self.size_bytes, self.duration_s, self.height, self.width):
427:     return (self.size_bytes, self.duration_s, self.height, self.width) # type:
ignore[return-value]
```

12. assistant (2026-07-06T23:17:18.459Z)

Now let me investigate the key concern: `asyncio.wait\_for` raises `TimeoutError`. In Python 3.11+, `asyncio.TimeoutError` IS `TimeoutError` (builtin), so the `except TimeoutError` catches it. Good. Let me verify the test for timeout and the semaphore concurrency, then run a repro on the config parsing edge cases.

13. user (2026-07-06T23:17:21.044Z)

```
diff --git a/tests/test_policy.py b/tests/test_policy.py
index 1dd660a..2ca3f22 100644
--- a/tests/test_policy.py
+++ b/tests/test_policy.py
@@ -2,6 +2,8 @@

from __future__ import annotations

+import asyncio
+
import aiosqlite

from tg_video_filter import db
@@ -9,8 +11,10 @@ from tg_video_filter.models import ContentItem, Policy, Source
from tg_video_filter.policy import (
    CaptionSafetyConfig,
    CaptionSafetyStage,
+    DeepInspectionStage,
    DomainReputationConfig,
    ExtractedUrl,
+    PolicyBudget,
    PolicyContext,
    StageResult,
    TopicPreferences,
@@ -31,6 +35,9 @@ def _ctx(
    policy_json: str = "{}",
    source_title: str | None = None,
    text: str | None = None,
+    size_bytes: int | None = None,
+    fetcher: object | None = None,
+    budget: PolicyBudget | None = None,
) -> PolicyContext:
    return PolicyContext(
        content_item=ContentItem(
@@ -38,6 +45,7 @@ def _ctx(
```

```

        source_id=1,
        provider_message_id="42",
        text=text,
        size_bytes=size_bytes,
        dedup_key="tg:msg:1:42",
    ),
    source=Source(
@@ -54,6 +62,8 @@ def _ctx(
        config_json=policy_json,
        is_default=True,
    ),
+    fetcher=fetcher, # type: ignore[arg-type]
+    budget=budget,
)

@@ -496,6 +506,209 @@ async def test_caption_safety_invalid_config_fails_open_with_explanation()
-> No
    assert result.metadata == {"invalid_caption_safety_config": True}

+## --- DeepInspectionStage -----
+
+
+class _FakeFetcher:
+    def __init__(self) -> None:
+        self.calls: list[tuple[ContentItem, PolicyBudget]] = []
+
+    async def inspect(self, content_item: ContentItem, budget: PolicyBudget) -> StageResult:
+        self.calls.append((content_item, budget))
+        return StageResult.pass_("deep-inspection", "deep inspection passed", inspected=True)
+
+
+class _SlowFetcher:
+    async def inspect(self, content_item: ContentItem, budget: PolicyBudget) -> StageResult:
+        await asyncio.sleep(1)
+        return StageResult.pass_("deep-inspection", "too late")
+
+
+class _ConcurrencyFetcher:
+    def __init__(self) -> None:
+        self.active = 0
+        self.max_active = 0
+
+    async def inspect(self, content_item: ContentItem, budget: PolicyBudget) -> StageResult:
+        self.active += 1
+        self.max_active = max(self.max_active, self.active)
+        try:
+            await asyncio.sleep(0.01)
+            return StageResult.pass_("deep-inspection", "deep inspection passed")
+        finally:
+            self.active -= 1
+
+
+async def test_deep_inspection_opt_out_passes_without_fetcher_or_budget() -> None:
+    fetcher = _FakeFetcher()
+
+    result = await DeepInspectionStage().evaluate(_ctx(fetcher=fetcher, size_bytes=1_000))
+
+    assert result.passed is True
+    assert result.reason == "deep inspection disabled"
+    assert fetcher.calls == []
+
+
+async def test_deep_inspection_missing_budget_fails_before_fetcher() -> None:

```



```

+     fetcher = _FakeFetcher()
+
+     result = await DeepInspectionStage().evaluate(
+         _ctx(policy_json='{"deep_inspect":true}', fetcher=fetcher, size_bytes=1_000)
+     )
+
+     assert result.passed is False
+     assert result.reason == "deep inspection budget missing"
+     assert result.metadata == {"missing_budget": True}
+     assert fetcher.calls == []
+
+
+async def test_deep_inspection_invalid_budget_fails_before_fetcher() -> None:
+     fetcher = _FakeFetcher()
+     budget = PolicyBudget(max_bytes_per_item=0, timeout_s=1.0, max_concurrent=1)
+
+     result = await DeepInspectionStage().evaluate(
+         _ctx(
+             policy_json='{"deep_inspect":true}',
+             fetcher=fetcher,
+             budget=budget,
+             size_bytes=1_000,
+         )
+     )
+
+     assert result.passed is False
+     assert result.reason == "deep inspection budget invalid"
+     assert result.metadata == {
+         "invalid_budget": True,
+         "max_bytes_per_item": 0,
+         "timeout_s": 1.0,
+         "max_concurrent": 1,
+     }
+     assert fetcher.calls == []
+
+
+async def test_deep_inspection_missing_size_fails_before_fetcher() -> None:
+     fetcher = _FakeFetcher()
+     budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=1.0, max_concurrent=1)
+
+     result = await DeepInspectionStage().evaluate(
+         _ctx(policy_json='{"deep_inspect":true}', fetcher=fetcher, budget=budget)
+     )
+
+     assert result.passed is False
+     assert result.reason == "deep inspection budget cannot be enforced without content size"
+     assert result.metadata == {
+         "missing_size_bytes": True,
+         "max_bytes_per_item": 2_000,
+     }
+     assert fetcher.calls == []
+
+
+async def test_deep_inspection_budget_exceeded_fails_before_fetcher() -> None:
+     fetcher = _FakeFetcher()
+     budget = PolicyBudget(max_bytes_per_item=999, timeout_s=1.0, max_concurrent=1)
+
+     result = await DeepInspectionStage().evaluate(
+         _ctx(
+             policy_json='{"deep_inspect":true}',
+             fetcher=fetcher,
+             budget=budget,
+             size_bytes=1_000,
+         )
+     )

```

```
+ assert result.passed is False
+ assert result.reason == "deep inspection budget exceeded"
+ assert result.metadata == {
+     "budget_exceeded": True,
+     "exceeded_budget": "max_bytes_per_item",
+     "size_bytes": 1_000,
+     "max_bytes_per_item": 999,
+ }
+ assert fetcher.calls == []
+
+
+
+
+async def test_deep_inspection_timeout_budget_fails_as_stage_result() -> None:
+    budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=0.001, max_concurrent=1)
+
+    result = await DeepInspectionStage().evaluate(
+        _ctx(
+            policy_json='{"deep_inspect":true}',
+            fetcher=_SlowFetcher(),
+            budget=budget,
+            size_bytes=1_000,
+        )
+    )
+
+    assert result.passed is False
+    assert result.reason == "deep inspection budget exceeded"
+    assert result.metadata == {
+        "budget_exceeded": True,
+        "exceeded_budget": "timeout_s",
+        "timeout_s": 0.001,
+    }
+
+
+
+
+async def test_deep_inspection_budget_limits_concurrency() -> None:
+    budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=1.0, max_concurrent=1)
+    fetcher = _ConcurrencyFetcher()
+    stage = DeepInspectionStage()
+
+    results = await asyncio.gather(
+        stage.evaluate(
+            _ctx(
+                policy_json='{"deep_inspect":true}',
+                fetcher=fetcher,
+                budget=budget,
+                size_bytes=1_000,
+            )
+        ),
+        stage.evaluate(
+            _ctx(
+                policy_json='{"deep_inspect":true}',
+                fetcher=fetcher,
+                budget=budget,
+                size_bytes=1_000,
+            )
+        ),
+    )
+
+    assert [result.passed for result in results] == [True, True]
+    assert fetcher.max_active == 1
+
+
+
+
+async def test_deep_inspection_missing_fetcher_fails_after_budget_passes() -> None:
+    budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=1.0, max_concurrent=1)
+
+    result = await DeepInspectionStage().evaluate(
```

```

+         _ctx(policy_json='{"deep_inspect":true}', budget=budget, size_bytes=1_000)
+     )
+
+     assert result.passed is False
+     assert result.reason == "deep inspection fetcher missing"
+     assert result.metadata == {"missing_fetcher": True}
+
+
+async def test_deep_inspection_valid_budget_delegates_to_fetcher() -> None:
+    fetcher = _FakeFetcher()
+    budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=1.0, max_concurrent=1)
+
+    result = await DeepInspectionStage().evaluate(
+        _ctx(
+            policy_json='{"deep_inspect":{"enabled":true}}',
+            fetcher=fetcher,
+            budget=budget,
+            size_bytes=1_000,
+        )
+    )
+
+    assert result.passed is True
+    assert result.reason == "deep inspection passed"
+    assert result.metadata == {"inspected": True}
+    assert [(item.size_bytes, call_budget) for item, call_budget in fetcher.calls] == [
+        (1_000, budget)
+    ]
+
+
+async def test_evaluate_policy_stages_uses_default_policy_config() -> None:
+    conn = await aiosqlite.connect(":memory:")

```

14. assistant (2026-07-06T23:17:26.523Z)

Now let me examine the critical bug candidate. The `PolicyBudget.inspect` wraps `fetcher.inspect` in `asyncio.wait_for(..., timeout=timeout_s)`. When the fetcher itself raises a real `TimeoutError` (not from `wait_for`), the stage's `except TimeoutError` would misclassify it as "budget exceeded". But more interesting: let me check the concurrency + timeout interaction and whether the semaphore is acquired *before* `wait_for`.

Let me focus on a concrete concern: the semaphore wait time counts against nothing ? but timeout only wraps the fetcher call, which is correct. Let me verify the `TimeoutError` catch works with a repro and test the config-parsing "stringly enabled" case.

15. assistant (2026-07-06T23:17:37.408Z)

```

import asyncio, json
from tg_video_filter.policy import (
    load_deep_inspection_enabled, PolicyBudget, DeepInspectionStage, PolicyContext,
)
from tg_video_filter.models import ContentItem, Policy, Source

def mkpolicy(cfg):
    return Policy(id=1, user_id=1, name="p", config_json=cfg, is_default=True)

```

1. Config parsing edge cases

```

cases = [
    '{"deep_inspect": true}',
    '{"deep_inspect": false}',
    '{"deep_inspect": {"enabled": true}}',
    '{"deep_inspect": {"enabled": "true"}}', # stringly typed
    '{"deep_inspect": "true"}',             # stringly typed top-level

```

```

{'deep_inspect': 1}', # int truthful
{'deep_inspect': {'enabled': 1}}, # int inside
{'deep_inspect': {}}, # empty dict -> enabled defaults False
{'deep_inspect': {'enabled': true, 'extra': 5}},
{''}, # missing key
]
for c in cases:
    try:
        r = load_deep_inspection_enabled(mkpolicy(c))
        print(f"{c!r:55} -> enabled={r}")
    except Exception as e:
        print(f"{c!r:55} -> RAISE {type(e).__name__}: {e}")

```

2. Does the stage catch a real TimeoutError from a fast fetcher (not wait_for)?

```

class RaisingFetcher:
    async def inspect(self, content_item, budget):
        raise TimeoutError("real error from downloader, not the budget")

async def main():
    budget = PolicyBudget(max_bytes_per_item=2000, timeout_s=10.0, max_concurrent=1)
    ctx = PolicyContext(
        content_item=ContentItem(id=1, source_id=1, provider_message_id="42",
                                size_bytes=1000, dedup_key="k"),
        source=Source(id=1, user_id=1, kind="channel", provider="telegram",
                     provider_ref="@x", is_active=True),
        policy=mkpolicy({'deep_inspect': true}),
        fetcher=RaisingFetcher(), budget=budget,
    )
    r = await DeepInspectionStage().evaluate(ctx)
    print("\nReal TimeoutError from fetcher ->", r.passed, repr(r.reason), r.metadata)

asyncio.run(main())

```

16. user (2026-07-06T23:17:37.410Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

17. user (2026-07-06T23:17:41.681Z)

```

class ContentItem(BaseModel):
    """A normalized content record discovered from any :class:`Source` (ADR 0011).

    Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
    (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
    NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
    ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
    wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    account_id: int = Field(description="FK ? accounts.id")
    source_id: int = Field(description="FK ? sources.id")
    provider: str = Field(default="telegram", description="Provider namespace")
    provider_message_id: str = Field(
        description="Provider-native message id (Telegram message_id, ?)"
    )
    content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
    text: str | None = Field(default=None, description="Caption / post text (nullable in Phase 1)")
    link_url: str | None = Field(default=None, description="For link/article posts (Phase 1: NULL)")

```

```

media_ref: str | None = Field(default=None, description="Provider media reference")
document_id: str | None = Field(
    default=None,
    description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
)
duration_s: int | None = None
width: int | None = None
height: int | None = None
size_bytes: int | None = None
mime_type: str | None = None
---SOURCE---
channel_id: int = Field(description="Origin Telegram channel id")
message_id: int = Field(description="Telegram message id within the channel")
text: str | None = Field(default=None, description="Caption / message text")
document_id: int | None = Field(
    default=None,
    description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
)

# Metadata extracted from the Telethon message (may be None if absent).
duration_s: int | None = None
width: int | None = None
height: int | None = None
size_bytes: int | None = None
mime_type: str | None = None

matched: bool = False
forwarded: bool = False
seen_at: datetime = Field(default_factory=_utcnow)

@property
def dedup_key(self) -> tuple[int, int, int]:
    """The natural key used for deduplication in the ``videos`` table."""
    return (self.user_id, self.channel_id, self.message_id)

@property
def content_fingerprint(self) -> tuple[int, int, int, int] | None:
    """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height, width).

    Returns None if any field is missing (cannot fingerprint).
    """
    if None in (self.size_bytes, self.duration_s, self.height, self.width):
        return None
    return (self.size_bytes, self.duration_s, self.height, self.width) # type:
ignore[return-value]

```

Default filter config shared by the bootstrap onboarding step.

18. assistant (2026-07-06T23:17:44.869Z)

ContentItem uses `account\_id` not `user\_id`, and Source/Policy fields differ. Let me check the actual field names for constructing the repro. Let me look at how the test constructs them (which I already have) and the Source/Policy model signatures.

19. user (2026-07-06T23:17:46.948Z)

```

123:class Source(BaseModel):
186:class Policy(BaseModel):

```

20. assistant (2026-07-06T23:17:50.852Z)

I'll base the repro on the exact fields the test uses (`\_ctx` builds ContentItem with account_id, source_id, provider_message_id, text, size_bytes, dedup_key; Source and Policy

fields visible in test diff). Let me write the repro reusing the test helper via import.

21. assistant (2026-07-06T23:18:02.363Z)

```
import asyncio, sys
from tg_video_filter.policy import (
    load_deep_inspection_enabled, PolicyBudget, DeepInspectionStage, PolicyContext,
)
from tg_video_filter.models import ContentItem, Policy, Source

def mkpolicy(cfg):
    return Policy(id=1, user_id=1, name="p", config_json=cfg, is_default=True)

def mkctx(cfg, fetcher=None, budget=None, size_bytes=1000):
    return PolicyContext(
        content_item=ContentItem(account_id=1, source_id=1, provider_message_id="42",
                                size_bytes=size_bytes, dedup_key="tg:msg:1:42"),
        source=Source(id=1, account_id=1, kind="channel", provider="telegram",
                     provider_ref="@x", is_active=True),
        policy=mkpolicy(cfg),
        fetcher=fetcher, budget=budget,
    )

cases = [
    '{"deep_inspect": true}',
    '{"deep_inspect": {"enabled": true}}',
    '{"deep_inspect": {"enabled": "true"}}',
    '{"deep_inspect": "true"}',
    '{"deep_inspect": 1}',
    '{"deep_inspect": {"enabled": 1}}',
    '{"deep_inspect": {}}',
    '{}',
]
print("=== config parsing ===")
for c in cases:
    try:
        r = load_deep_inspection_enabled(mkpolicy(c))
        print(f"{c!r:45} -> enabled={r}")
    except Exception as e:
        print(f"{c!r:45} -> RAISE {type(e).__name__}: {e}")

class RaisingFetcher:
    async def inspect(self, content_item, budget):
        raise TimeoutError("real error from downloader internals, NOT the budget")

async def main():
    budget = PolicyBudget(max_bytes_per_item=2000, timeout_s=10.0, max_concurrent=1)
    ctx = mkctx('{"deep_inspect": true}', fetcher=RaisingFetcher(), budget=budget)
    r = await DeepInspectionStage().evaluate(ctx)
    print("\n=== real TimeoutError from fetcher ===")
    print("passed=", r.passed, "reason=", repr(r.reason), "meta=", r.metadata)

    # invalid-config stringly enabled: fail-OPEN?
    print("\n=== stringly enabled goes through stage ===")
    ctx2 = mkctx('{"deep_inspect": {"enabled": "true"}}', fetcher=RaisingFetcher(),
                 budget=budget)
    r2 = await DeepInspectionStage().evaluate(ctx2)
    print("passed=", r2.passed, "reason=", repr(r2.reason), "meta=", r2.metadata)

asyncio.run(main())
```

22. user (2026-07-06T23:18:02.365Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

23. assistant (2026-07-06T23:18:04.193Z)

The file exists from a prior partial write. Let me read then write.

24. user (2026-07-06T23:18:06.369Z)

```
1     import asyncio
2     from tg_video_filter.policy import PolicyBudget, StageResult
3
4     b = PolicyBudget(max_bytes_per_item=100, timeout_s=1.0, max_concurrent=1)
5     print("semaphore created sync, no running loop. _semaphore =", b._semaphore)
6
7     class SlowFetcher:
8         def __init__(self):
9             self.active = 0
10            self.max_active = 0
11        async def inspect(self, content_item, budget):
12            self.active += 1
13            self.max_active = max(self.max_active, self.active)
14            await asyncio.sleep(0.05)
15            self.active -= 1
16            return StageResult.pass_("deep-inspection", "ok")
17
18    class Item:
19        size_bytes = 10
20
21    async def main():
22        shared = PolicyBudget(max_bytes_per_item=100, timeout_s=5.0, max_concurrent=1)
23        f = SlowFetcher(); item = Item()
24        await asyncio.gather(*[shared.inspect(f, item) for _ in range(5)])
25        print("SHARED budget max_concurrent=1 -> observed max_active =", f.max_active)
26
27        f2 = SlowFetcher()
28        budgets = [PolicyBudget(max_bytes_per_item=100, timeout_s=5.0, max_concurrent=1) for
_ in range(5)]
29        await asyncio.gather(*[bd.inspect(f2, item) for bd in budgets])
30        print("SEPARATE budgets each max_concurrent=1 -> observed max_active =",
f2.max_active)
31
32        try:
33            await b.inspect(f, item)
34            print("sync-created semaphore works under running loop: OK")
35        except Exception as e:
36            print("sync-created semaphore FAILED under running loop:", type(e).__name__, e)
37
38        cancelled = {"v": False}
39        class HangFetcher:
40            async def inspect(self, content_item, budget):
41                try:
42                    await asyncio.sleep(100)
43                except asyncio.CancelledError:
44                    cancelled["v"] = True
45                raise
46            return StageResult.pass_("deep-inspection", "ok")
47        tb = PolicyBudget(max_bytes_per_item=100, timeout_s=0.05, max_concurrent=2)
48        try:
49            await tb.inspect(HangFetcher(), item)
50        except TimeoutError:
51            pass
52        await asyncio.sleep(0.01)
53        print("inner coroutine cancelled on timeout:", cancelled["v"])
```

```

54         print("semaphore permits after timeout (should be 2):", tb._semaphore._value)
55
56     asyncio.run(main())
57

```

25. assistant (2026-07-06T23:18:09.220Z)

There's already a useful repro here from a prior session probing important concerns: sync-created semaphore under a running loop, semaphore permit release after timeout (cancellation). Let me run this one first, then run the config-parsing repro.

26. user (2026-07-06T23:18:14.384Z)

```

semaphore created sync, no running loop. _semaphore = <asyncio.locks.Semaphore object at
0x0000001BD522E7CB0 [unlocked, value:1]>
SHARED budget max_concurrent=1 -> observed max_active = 1
SEPARATE budgets each max_concurrent=1 -> observed max_active = 5
sync-created semaphore works under running loop: OK
inner coroutine cancelled on timeout: True
semaphore permits after timeout (should be 2): 2

```

27. assistant (2026-07-06T23:18:18.385Z)

Key results:

- Sync-created semaphore works fine under a running loop (Python 3.10+ semaphores don't bind to a loop at construction). Good, no bug there.
- On timeout, the inner coroutine IS cancelled and the semaphore permit IS released. Good ? no permit leak.
- `max\_concurrent=1` on a SHARED budget correctly limits to 1. Separate budgets each allow 1 (5 total) ? expected since each PolicyBudget has its own semaphore.

Now the config-parsing repro. Let me write it to a new file.

28. assistant (2026-07-06T23:18:28.368Z)

```

import asyncio
from tg_video_filter.policy import (
    load_deep_inspection_enabled, PolicyBudget, DeepInspectionStage, PolicyContext,
)
from tg_video_filter.models import ContentItem, Policy, Source

def mkpolicy(cfg):
    return Policy(id=1, user_id=1, name="p", config_json=cfg, is_default=True)

def mkctx(cfg, fetcher=None, budget=None, size_bytes=1000):
    return PolicyContext(
        content_item=ContentItem(account_id=1, source_id=1, provider_message_id="42",
                                size_bytes=size_bytes, dedup_key="tg:msg:1:42"),
        source=Source(id=1, account_id=1, kind="channel", provider="telegram",
                     provider_ref="@x", is_active=True),
        policy=mkpolicy(cfg),
        fetcher=fetcher, budget=budget,
    )

cases = [
    '{"deep_inspect": true}',
    '{"deep_inspect": {"enabled": true}}',
    '{"deep_inspect": {"enabled": "true"}}',
    '{"deep_inspect": "true"}',
    '{"deep_inspect": 1}',
    '{"deep_inspect": {"enabled": 1}}',

```



```

    '{"deep_inspect": {}}',
    '{}',
]
print("=== config parsing ===")
for c in cases:
    try:
        r = load_deep_inspection_enabled(mkpolicy(c))
        print(f"{c!r:45} -> enabled={r}")
    except Exception as e:
        print(f"{c!r:45} -> RAISE {type(e).__name__}: {e}")

class RaisingFetcher:
    async def inspect(self, content_item, budget):
        raise TimeoutError("real error from downloader internals, NOT the budget")

async def main():
    budget = PolicyBudget(max_bytes_per_item=2000, timeout_s=10.0, max_concurrent=1)
    ctx = mkctx('{"deep_inspect": true}', fetcher=RaisingFetcher(), budget=budget)
    r = await DeepInspectionStage().evaluate(ctx)
    print("\n=== real TimeoutError raised by fetcher internals ===")
    print("passed=", r.passed, "reason=", repr(r.reason), "meta=", r.metadata)

    print("\n=== stringly enabled -> fail-open (pass) ===")
    ctx2 = mkctx('{"deep_inspect": {"enabled": "true"}}', fetcher=RaisingFetcher(),
budget=budget)
    r2 = await DeepInspectionStage().evaluate(ctx2)
    print("passed=", r2.passed, "reason=", repr(r2.reason), "meta=", r2.metadata)

asyncio.run(main())

```

29. user (2026-07-06T23:18:28.368Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

30. assistant (2026-07-06T23:18:30.628Z)

Another stale file exists. Let me use a fresh unique name.

31. user (2026-07-06T23:18:43.665Z)

```

Exit code 1
=== config parsing ===
'{"deep_inspect": true}' -> RAISE ValidationError: 1 validation error for
Policy
account_id
  Field required [type=missing, input_value={'id': 1, 'user_id': 1, '...e'}, 'is_default':
True], input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
'{"deep_inspect": {"enabled": true}}' -> RAISE ValidationError: 1 validation error for
Policy
account_id
  Field required [type=missing, input_value={'id': 1, 'user_id': 1, '...}}, 'is_default':
True], input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
'{"deep_inspect": {"enabled": "true"}}' -> RAISE ValidationError: 1 validation error for
Policy
account_id
  Field required [type=missing, input_value={'id': 1, 'user_id': 1, '...}}, 'is_default':
True], input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
'{"deep_inspect": "true"}' -> RAISE ValidationError: 1 validation error for
Policy

```

```

account_id
    Field required [type=missing, input_value={'id': 1, 'user_id': 1, '...'}, 'is_default':
True], input_type=dict]
    For further information visit https://errors.pydantic.dev/2.13/v/missing
'{"deep_inspect": 1}' -> RAISE ValidationError: 1 validation error for
Policy
account_id
    Field required [type=missing, input_value={'id': 1, 'user_id': 1, '...1'}, 'is_default':
True], input_type=dict]
    For further information visit https://errors.pydantic.dev/2.13/v/missing
'{"deep_inspect": {"enabled": 1}}' -> RAISE ValidationError: 1 validation error for
Policy
account_id
    Field required [type=missing, input_value={'id': 1, 'user_id': 1, '...'}], 'is_default':
True], input_type=dict]
    For further information visit https://errors.pydantic.dev/2.13/v/missing
'{"deep_inspect": {}}' -> RAISE ValidationError: 1 validation error for
Policy
account_id
    Field required [type=missing, input_value={'id': 1, 'user_id': 1, '...'}], 'is_default':
True], input_type=dict]
    For further information visit https://errors.pydantic.dev/2.13/v/missing
'{}' -> RAISE ValidationError: 1 validation error for
Policy
account_id
    Field required [type=missing, input_value={'id': 1, 'user_id': 1, '...{'}], 'is_default':
True], input_type=dict]
    For further information visit https://errors.pydantic.dev/2.13/v/missing
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro_cfg.py", line 44, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro_cfg.py", line 35, in main
    ctx = mkctx('{"deep_inspect": true}', fetcher=RaisingFetcher(), budget=budget)
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro_cfg.py", line 14, in mkctx
    source=Source(id=1, account_id=1, kind="channel", provider="telegram",
    ~~~~~^~~~~~
    provider_ref="@x", is_active=True),
    ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in
__init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 1 validation error for Source
provider_source_id
    Field required [type=missing, input_value={'id': 1, 'account_id': 1...'@x', 'is_active':
True}, input_type=dict]
    For further information visit https://errors.pydantic.dev/2.13/v/missing

```

The Policy needs `account\_id` not `user\_id`. The config parsing only depends on `config\_json`, so I can construct a minimal Policy directly. Let me check the exact required fields for Policy.

33. user (2026-07-06T23:18:47.729Z)

```
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account ``filter_config`` row so one account can
190         own multiple policies (enables ADR 0009's per-route filter overrides) and so
191         future enrichment/security stages grow into the same ``config_json`` column.
192         Exactly one policy per account carries ``is_default=True``; the legacy
193         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194         """
195
196         model_config = ConfigDict(frozen=True)
197
198         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199         account_id: int = Field(description="FK ? accounts.id")
200         name: str = Field(description="Policy name; 'default' for the account default
policy")
201         config_json: str = Field(
202             description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203         )
204         is_default: bool = Field(
205             default=False, description="Exactly one per account (UNIQUE partial index)"
206         )
207         created_at: datetime = Field(default_factory=_utcnow)
208         updated_at: datetime = Field(default_factory=_utcnow)
209
210
211     # --- Track C: destinations, routing, delivery state (ADR 0011) -----
212     #
213     # These tables were created empty by Track B (_SCHEMA); Track C populates
214     # destinations (backfilled from filter_config_deprecated.config_json), writes
215     # delivery_attempts (replacing the legacy videos.forwarded boolean), and
216     # defines the models for per-destination delivery state. The source/destination
217     # invariant (a channel is either source or destination, never both) is enforced
218     # at the app level by future selective-scanning logic (Track E, #30).
219
220
221     class Destination(BaseModel):
222         """A delivery target ? where matched content is sent (ADR 0011, Track C).
223
224         Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
225         can have multiple named output channels. Backfilled from the existing
```

34. assistant (2026-07-06T23:18:50.410Z)

Policy needs `account\_id`, `name`, `config\_json`. Let me fix the config-parsing repro (which only needs Policy).

35. user (2026-07-06T23:18:59.568Z)

```
'{"deep_inspect": true}'          -> enabled=True
'{"deep_inspect": {"enabled": true}}' -> enabled=True
'{"deep_inspect": {"enabled": "true"}}' -> RAISE ValueError: deep_inspect must be a
boolean or an object with boolean enabled
'{"deep_inspect": "true"}'        -> RAISE ValueError: deep_inspect must be a
boolean or an object with boolean enabled
'{"deep_inspect": 1}'             -> RAISE ValueError: deep_inspect must be a
boolean or an object with boolean enabled
'{"deep_inspect": {"enabled": 1}}' -> RAISE ValueError: deep_inspect must be a
boolean or an object with boolean enabled
```

```
'{"deep_inspect": {}}' -> enabled=False
'{"deep_inspect": null}' -> RAISE ValueError: deep_inspect must be a
boolean or an object with boolean enabled
'{"deep_inspect": {"enabled": null}}' -> RAISE ValueError: deep_inspect must be a
boolean or an object with boolean enabled
'{}' -> enabled=False
```

36. assistant (2026-07-06T23:19:03.211Z)

Config parsing confirmed. Now the important part: what happens in the STAGE when config is stringly-typed (raises ValueError ? fail-OPEN pass). This is the semantic question. And the real-TimeoutError misclassification. Let me run that part with correct model fields. Let me get Source's exact fields.

37. user (2026-07-06T23:19:04.135Z)

```
123 class Source(BaseModel):
124     """A configured place we ingest content from (ADR 0011).
125
126     Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127     providers add their own source kinds). ``provider_source_id`` is the
128     provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129     """
130
131     model_config = ConfigDict(frozen=True)
132
133     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134     account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135     provider: str = Field(default="telegram", description="Provider namespace")
136     provider_source_id: str = Field(
137         description="Provider-native source id (Telegram channel_id, ?)"
138     )
139     title: str | None = None
140     kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141     watched: bool = Field(
142         default=False,
143         description="ADR 0009 selective scanning: ingest from this source iff True",
144     )
145     seen_first_at: datetime = Field(default_factory=_utcnow)
146     last_seen_at: datetime | None = None
147
148
```

38. user (2026-07-06T23:19:18.677Z)

deep inspection disabled due to invalid config policy_id=None

Traceback (most recent call last):

```
File "C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py", line 681, in evaluate
    enabled = load_deep_inspection_enabled(ctx.policy)
File "C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py", line 306, in
load_deep_inspection_enabled
    raise ValueError("deep_inspect must be a boolean or an object with boolean enabled")
ValueError: deep_inspect must be a boolean or an object with boolean enabled
=== fetcher raises real TimeoutError (fast, well within budget) ===
passed= False | reason= 'deep inspection budget exceeded' | meta= {'budget_exceeded': True,
'exceeded_budget': 'timeout_s', 'timeout_s': 10.0}
```

=== stringly enabled -> stage result ===

```
passed= True | reason= 'invalid deep inspection config ignored: deep_inspect must be a boolean
or an object with boolean enabled' | meta= {'invalid_deep_inspection_config': True}
```

39. assistant (2026-07-06T23:19:27.718Z)

Both confirmed:

1. ****TimeoutError misclassification**** ? a real ``TimeoutError`` raised by the fetcher internals (completing well within the 10s budget) is reported as ``budget_exceeded`/`exceeded_budget=timeout_s`` with the wrong ``timeout_s=10.0``. This mislabels a genuine fetcher error as a budget-timeout event. Direction is still fail-closed (drop), so it is not a bypass ? this is a medium correctness/observability defect.

2. ****Stringly-enabled**** ? ``deep_inspect={"enabled":"true"}`` raises `ValueError` ? fail-OPEN pass. So an operator who typos a quoted boolean silently gets deep inspection NOT enforced. Given the whole feature is dormant/fail-closed-when-actually-enabled, this is a low/note about the open-vs-closed split coherence.

Now let me verify the remaining dimensions: stage order/short-circuit, that ``deep_inspect`` is a distinct top-level key from F2/F3/F4 configs (no collision), and check the ADR/docs for documentation of the operational footgun. Let me look at ``evaluate_pipeline`` and check config keys used by other stages.

40. user (2026-07-06T23:19:30.493Z)

```
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    """Evaluate stages in order and short-circuit on the first failure."""
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results
    return True, results
```

```
__all__ = [
    "CaptionSafetyConfig",
    "CaptionSafetyStage",
    "ContentFetcher",
    "DeepInspectionStage",
    """ config keys used by loaders """
181:     raw_topics = config.get("topics")
186:     include = _string_tuple(raw_topics.get("include"), field_name="topics.include")
187:     exclude = _string_tuple(raw_topics.get("exclude"), field_name="topics.exclude")
190:     mode = raw_topics.get("mode", "any")
206:     raw_security = config.get("security_checks")
213:         raw_security.get("unsafe_caption_terms"),
217:         raw_security.get("unsafe_caption_patterns"),
243:     raw_security = config.get("security_checks")
250:         raw_security.get("blocked_domains"),
299:     raw_deep_inspect = decoded.get("deep_inspect", False)
303:         enabled = raw_deep_inspect.get("enabled", False)
```

41. assistant (2026-07-06T23:19:34.566Z)

Config keys confirmed non-colliding: F2 uses ``topics``, F3/F4 use ``security_checks``, F5 uses top-level ``deep_inspect``. No collision. ``evaluate_pipeline`` short-circuits on first failure, and `DeepInspectionStage` is last ? correct short-circuit ordering (cheapest first). Default (unset ``deep_inspect``) ? ``enabled=False`` ? immediate pass ? confirmed no-op.

Now let me check the ADR/docs for documentation of the operational footgun (enabling `deep_inspect` today drops all matched videos).

42. user (2026-07-06T23:19:36.840Z)

0001-userbot-over-botapi.md
0010-identity-and-session-separation.md

0011-provider-neutral-domain-schema.md
0012-policy-pipeline.md
docs/adr/0012-policy-pipeline.md
docs/design.md
docs/plan.md

43. user (2026-07-06T23:19:40.593Z)

```
65:? Tier 4 ? Deep inspection (opt-in, heavy I/O) ?
67:? ? gated behind explicit user opt-in + budget ?
84: budget: PolicyBudget | None = None # bytes/time/concurrency limits for I/O
101:`ctx.budget`.
157:### Deep inspection ? Tier 4
159:Deep inspection (ffprobe, perceptual hash, downloaded content scan) is
161:no-download principle. The opt-in is per-policy (`deep_inspect: true` in
162:`config_json`) and the stage has a **budget** (max bytes per item, max
174:- The policy explicitly opts in (`deep_inspect: true`).
177: `PolicyBudget`.
216: `deep_inspect` keys.
219: serializes `FilterConfig` again, so unknown keys would be dropped. F1 must
235:| **F5** ? Deep inspection | `ContentFetcher`, ffprobe/perceptual hash, download budget
enforcement | F1, accepted ADR 0003 amendment above |
261: making Tier 4's fetcher/budget explicit.
263: F1 must avoid dropping pipeline keys when legacy config commands rewrite
270:- Deep inspection amends ADR 0003 and must be explicitly opt-in with a budget
299:4. **Deep inspection budget** ? per-item byte limit or per-account
```

44. user (2026-07-06T23:19:43.156Z)

```
157    ### Deep inspection ? Tier 4
158
159    Deep inspection (ffprobe, perceptual hash, downloaded content scan) is
160    **gated behind explicit opt-in** because it violates ADR 0003's
161    no-download principle. The opt-in is per-policy (`deep_inspect: true` in
162    `config_json`) and the stage has a **budget** (max bytes per item, max
163    concurrent downloads) enforced by the policy engine.
164
165    This tier is explicitly deferred to a follow-up issue ? the design only
166    **reserves the slot** in the pipeline model so it doesn't require a schema
167    migration when implemented.
168
169    #### ADR 0003 amendment for Tier 4
170
171    If this ADR is accepted, it amends ADR 0003 only for Tier 4: downloading or
172    probing content is allowed when all of the following are true:
173
174    - The policy explicitly opts in (`deep_inspect: true`).
175    - The item has already passed Tiers 1-3.
176    - The stage receives a bounded `ContentFetcher` and an enforced
177      `PolicyBudget`.
178    - The result is recorded as a policy decision, not as a delivery failure.
179
180    All earlier tiers continue to honor ADR 0003's metadata-first/no-download
181    default.
182
183    ### Policy evaluation flow (post-dedup, pre-delivery)
184
185    ```python
186    async def evaluate_pipeline(
```

45. assistant (2026-07-06T23:19:47.778Z)

The ADR documents the design but does NOT document the operational footgun that enabling
`deep\_inspect: true` today (with live wiring passing `fetcher=None, budget=None`) causes ALL

matched videos to be DROPPED. There is no comment in `telethon\_client.py` at the wiring site warning that the stage is dormant/fail-closed, and no comment/guard preventing a future `/set deep\_inspect` command from shipping before fetcher/budget wiring. Let me verify there is no such comment at the wiring site.

46. user (2026-07-06T23:19:49.335Z)

```
async def _evaluate_policy_stages(
    conn: aiomysqlite.Connection,
    account_id: int,
    source: Source,
    content_item: ContentItem,
) -> tuple[bool, list[StageResult]]:
    """Evaluate ADR 0012 policy stages for a candidate item."""
    policy = await db.get_default_policy(conn, account_id)
    if policy is None:
        return True, []
    return await evaluate_pipeline(
        PolicyContext(content_item=content_item, source=source, policy=policy),
        [
            TopicPreferenceStage(),
            UrlReputationStage(),
            CaptionSafetyStage(),
            DeepInspectionStage(),
        ],
    )
```

```
def _peer_id(peer: object) -> int:
    """Best-effort numeric id for the peer of a message.
```

Telegram represents channel/group/user peers as different TL types; we

47. assistant (2026-07-06T23:19:56.903Z)

Confirmed: no warning comment at the wiring site. The stage is wired live with `fetcher=None, budget=None`, meaning any policy with `deep\_inspect: true` drops every matched video with reason "deep inspection budget missing". This is dormant only because no command sets `deep\_inspect`. This is a documented note.

Let me also double-check one subtle thing: the `except TimeoutError` in the stage ? since `\_deep\_inspection\_budget\_result` runs the size check before calling the fetcher, and the fetcher call is wrapped in `wait\_for`. But note the semaphore acquisition happens INSIDE `budget.inspect` and is NOT covered by `wait\_for`'s timeout ? if the semaphore is contended, an item can wait indefinitely for a permit (the timeout only starts after acquiring the permit). Let me verify this against the code and assess. Also check: `wait\_for(timeout=timeout\_s)` ? the timeout wraps only `fetcher.inspect`, semaphore acquire is outside. Let me confirm with a repro.

48. user (2026-07-06T23:20:09.174Z)

```
A(holds permit): TimeoutError after 0.11s
B(waits for permit): TimeoutError after 0.22s
```

49. assistant (2026-07-06T23:20:17.991Z)

Interesting result: B waited ~0.11s for A to release the permit (A timed out at 0.11s, releasing it), then B acquired it and immediately timed out at its own 0.1s. So the semaphore-wait is NOT bounded by the budget timeout ? B's total wall time was 0.22s (0.11s waiting + 0.1s running).

To make the unbounded-wait risk concrete: if A's fetcher were a genuinely long download and A's `timeout\_s` were large, B would block for the FULL duration A holds the permit, regardless of B's own timeout. This means the effective time budget per item is `timeout\_s`, but the queueing delay to acquire a permit is unbounded (up to `max\_concurrent` items × `timeout\_s` ahead in line). This is a design characteristic of budget enforcement, not a bypass ? the per-download

work is still bounded. It's a latent design note (medium/low) worth flagging since the timeout does not bound total time-to-decision under contention. Given the feature is dormant, I'll rank it low-medium.

Let me confirm the gate is green (per reviewer, 543 passed) by running the policy tests quickly to make sure the diff's tests pass in this checkout.

50. user (2026-07-06T23:20:22.568Z)

src\tg_video_filter\bootstrap.py	23	23	0	0	0%	17-78
src\tg_video_filter\command_service.py	218	177	94	0	13%	53-61, 66-71, 78-94, 98-107, 112-126, 131, 215-343, 351-361, 367-374, 378-391, 395-406, 416-431, 449-478
src\tg_video_filter\commands.py	46	30	12	0	28%	58-88, 111-113, 128-131
src\tg_video_filter\config.py	26	5	2	0	75%	62-64, 69, 79
src\tg_video_filter\db.py	352	270	56	0	20%	199-200, 206-221, 296-316, 321-329, 334-336, 346-350, 377-505, 523-668, 685-742, 769-772, 780, 792-795, 803, 813, 832-839, 848, 862, 885, 897, 910, 923, 942-958, 962-965, 972-975, 982-1010, 1016-1021, 1034-1043, 1070-1082, 1087-1091, 1101-1102, 1124-1152, 1159-1166, 1180-1207, 1217-1224, 1236-1242, 1256-1281, 1286-1292, 1302-1305, 1310, 1320-1323, 1330-1331, 1350-1377, 1384-1390, 1408-1431, 1438-1444, 1457-1469, 1474-1475, 1486-1491, 1512-1516, 1523-1530, 1555-1571, 1595-1598, 1605-1609, 1616-1624, 1638-1653, 1664-1702, 1717, 1730, 1764-1767, 1778-1793, 1803-1813, 1887
src\tg_video_filter\delivery.py	102	80	48	0	15%	40-48, 53-58, 63-65, 87-95, 107-121, 145-160, 177-192, 218-278
src\tg_video_filter\filters__init__.py	39	26	18	0	23%	32-33, 42-52, 62-75
src\tg_video_filter\filters\base.py	26	2	0	0	92%	48, 52
src\tg_video_filter\filters\content_type.py	23	15	12	0	23%	18-22, 29-33, 40-44
src\tg_video_filter\filters\duration.py	15	10	6	0	24%	17-18, 21-28
src\tg_video_filter\filters\resolution.py	13	8	6	0	26%	16, 21-27
src\tg_video_filter\filters\size.py	18	11	6	0	29%	18, 21-29, 34
src\tg_video_filter\manager.py	36	36	8	0	0%	26-204
src\tg_video_filter\models.py	163	10	8	1	90%	363-367, 377, 417, 425-427
src\tg_video_filter\policy.py	376	229	144	5	32%	58->exit, 64, 77, 124, 137, 152, 156, 168, 181-193, 206-232, 243-264, 272-275, 280-283, 288-291, 298, 306, 310-321, 330-376, 384-385, 389, 394-423, 428-439, 443-446, 452, 461-513, 524-532, 541-636, 655-671, 682-688, 762-768
src\tg_video_filter\telethon_client.py	79	49	32	1	28%	81-119, 361, 374-379, 384-390, 403-404, 438-441, 460-465, 477->exit

TOTAL	1612	1037	468	7	29%	

FAIL Required test coverage of 80.0% not reached. Total coverage: 28.75%
9 passed, 33 deselected in 1.23s

51. assistant (2026-07-06T23:20:31.481Z)

All 9 deep-inspection tests pass (the coverage failure is just because I filtered to a subset). The reviewer already confirmed the full gate at 543 passed / 94.57%. My findings are all verified. Let me compile the report.

Summary of my analysis across the assigned dimension:

- **Config parsing**: ``deep_inspect`` accepts ``bool`` or `{enabled: bool}`; any other shape (stringly-typed ``true``, `int `1``, ``null``) raises `ValueError` ? stage fails OPEN (pass). Confirmed via repro.
- **Fail posture split**: `parse-error` ? `fail-open`; `asked-but-can't-run` (`enabled + no budget`) ? `fail-closed`. This split is intentional but the stringly-typed `enable` silently does NOT enable and is not caught anywhere.
- **Wiring**: `live `_evaluate_policy_stages`` passes ``fetcher=None, budget=None`` ? any ``deep_inspect: true`` drops ALL matched videos. Dormant (no command sets it) but undocumented at the wiring site.
- **TimeoutError misclassification**: a real ``TimeoutError`` from `fetcher` internals is relabeled

as budget-exceeded/timeout. Confirmed via repro.

- ****Semaphore wait not bounded by timeout****: confirmed via repro; per-item work is bounded but queueing delay is not.
- ****No config collision**** with F2/F3/F4 (`topics`/`security\_checks` vs top-level `deep\_inspect`). Stage order/short-circuit correct; default is a no-op pass. No permit leak on timeout (inner coroutine cancelled).

52. user (2026-07-06T23:21:09.940Z)

Structured output provided successfully